

Backend Feature List — Complete

Every backend feature to build for VAYU-SUCHAK 2.0, including the self-trained AI models. Architecture is **live-first**: real headless-browser scraping of airline and OTA portals is the primary path and the core differentiator. Travelpayouts (Aviasales) data API is a formal fallback tier; an on-disk cache snapshot is the last resort. No external AI APIs — all models are trained in-house on free datasets.

BACKEND **FastAPI** · one process · port **8000** MODULES **12** AI MODELS **2 trained + 1 rule-based**

EXTERNAL AI APIs **none** COST **₹0**

00 Infra & App Shell

#	FEATURE	DETAIL
0.1	FastAPI single-process app	port 8000, serves static frontend + API, one process, no Flask
0.2	<code>uvicorn</code> ASGI server	async — SSE stream and scrape run concurrently in one request
0.3	Config layer	<code>.env</code> + <code>pydantic-settings</code> → <code>k_factor</code> , dates, <code>PG_DSN</code> , <code>TRAVELPAYOUTS_TOKEN</code> , rate limits — all parameters, nothing hardcoded
0.4	<code>AuditParams</code> Pydantic model	corridor regex <code>^[A-Z]{3}-[A-Z]{3}\$</code> , date range ≤ 90d, <code>k_factor</code> 0–10, end ≥ start → bad input = 422 before pipeline runs
0.5	Async orchestrator (<code>app/orchestrator.py</code>)	async generator, yields 1 SSE line per stage, <code>sleep(0.2)</code> for visible streaming, <code>run_id</code> = uuid4
0.6	<code>pipeline/</code> = pure functions	no FastAPI/DB/logging inside — <code>(data) → (result, summary)</code> , testable from a shell
0.7	Structured error handling	every stage try/except, partial failure → pipeline continues in degraded mode, no silent crash

01 Live Acquisition — the differentiator

Real headless browsers scrape 5 airline / OTA portals live. Everything is live — that is what sets the project apart. When a source is blocked, the Travelpayouts data API fills that route; only when fully offline does the on-disk cache serve.

1A Live Scraper Pool — `pipeline/scrape/`

#	FEATURE	DETAIL
1.1	Per-source Playwright adapter	MakeMyTrip, Ixigo, Cleartrip, IndiGo, Air India — one lightweight adapter class each
1.2	Headless Chromium pool	browser instance reuse, new context/page per adapter, concurrency cap (e.g. 3 parallel)
1.3	Concurrent fan-out	<code>asyncio.gather</code> across adapters × dates, merge results
1.4	XHR intercept (primary extraction)	<code>page.on("response")</code> → capture the site's own fare JSON, no HTML parsing. Each adapter has <code>is_fare_xhr()</code> + <code>parse_xhr()</code>
1.5	CSS-selector fallback extraction	XHR shape missing → parse rendered fare cards from the DOM (Playwright locators). Vision-LLM path removed.
1.6	Normalise contract	<code>{flight_id, route, price, currency, departure_ts, airline, source}</code> — every later module depends on this
1.7	Merge + dedupe	key = <code>airline + flight_no + departure_ts</code> , duplicate count logged
1.8	Per-domain rate limiter	1 req / few sec per domain, token-bucket, configurable
1.9	<code>robots.txt</code> parser + honour	fetch, cache, respect disallow, honour <code>Retry-After</code>
1.10	Rotating User-Agent	pool of realistic UAs, per-session rotation, realistic headers / viewport
1.11	Anti-bot / captcha detection	403 / captcha page / block signature detected → mark source failed, trigger fallback
1.12	Off-peak scheduler	APScheduler / cron — bulk pre-scrape overnight, keeps cache warm
1.13	Per-source status report	<code>{source: {status, fares, latency_ms, extraction_path}}</code> → feeds console + status strip
1.14	Timeout + retry with backoff	per-page timeout, N retries with exponential backoff, then fallback

1B Fallback Chain — 3-tier resilience

#	FEATURE	DETAIL
1.15	Tier 1 — Travelpayouts API fallback	live scrape fails / blocked → <code>api.travelpayouts.com</code> (Aviasales data: <code>prices_for_dates</code> / cheapest fares) for that route, mapped into the same normalise contract. Token in <code>.env</code> .
1.16	Travelpayouts client	<code>httpx.AsyncClient</code> , auth header, response → <code>list[dict]</code> in record shape, currency normalised to INR

#	FEATURE	DETAIL
1.17	Tier 2 — on-disk cache snapshot	Travelpayouts also unreachable (offline) → newest cached JSON for <code>source_corridor_date</code>
1.18	Cache writer	every successful scrape / API response written to disk, timestamped, <code>data/cache/</code>
1.19	Cache seed script	Kaggle dataset → 30–50 rows/corridor as initial cache (record shape) so the pipeline runs on day one
1.20	Fallback tagging	each record + run tagged <code>ingestion_source</code> = <code>live_scrape</code> / <code>travelpayouts</code> / <code>cache</code> , console emits <code>[SCRAPE]</code> <code>[FALLBACK]</code>
1.21	Graceful degradation	1 source blocked → other 4 + Travelpayouts still contribute; run completes, <code>sources_degraded</code> count reported

02 Festival Flag Engine — `pipeline/festival.py`

#	FEATURE	DETAIL
2.1	<code>holidays.India</code> date builder	<code>holidays.India(years=[2026, 2027])</code> , filter to major festivals (Diwali, Holi, Dussehra, Christmas, Guru Nanak Jayanti, Eid al-Fitr, Eid al-Adha)
2.2	±7 day window flagging	each record: <code>is_festival_season</code> (0/1) + <code>festival_matched</code> (name), <code>window_days</code> config param
2.3	Flagged count + breakdown	<code>{Diwali: 31, Christmas: 7}</code> per festival; festivals outside the window do not appear
2.4	Date-verification test	<code>test_festival.py</code> — assert 2026 dates against a real calendar

03 Dynamic IQR Purifier — `pipeline/iqr.py`

#	FEATURE	DETAIL
3.1	Two-tailed Tukey fence	<code>Q1 - k·IQR</code> and <code>Q3 + k·IQR</code> — catches both glitch-low (₹1, null-as-0) and absurd-high
3.2	<code>clean[]</code> / <code>anomalies[]</code> split	driven by <code>k_factor</code> ; price within fence → clean, else anomaly
3.3	Exclusion metadata	each anomaly gets <code>exclusion_reason</code> (<code>below_lower_fence</code> / <code>above_upper_fence</code>) + the fence value
3.4	Fences object returned	<code>{Q1, Q3, IQR, lower_fence, upper_fence}</code> → feeds histogram + console
3.5	Nothing deleted	anomalies logged + stored + returned — any <code>k</code> choice is reversible
3.6	Golden-number test	<code>test_iqr.py</code> — worked example: <code>k=1.5</code> → 2 excluded, <code>k=3.0</code> → 1

04 AI · Data-Integrity Model — `pipeline/ml/integrity.py`

Trained in-house. No external API. Runs on top of the IQR filter as a second, learned pass.

#	FEATURE	DETAIL
4.1	Isolation Forest anomaly detector	<code>sklearn.ensemble.IsolationForest</code> — flags decoy / placeholder / bot-trap fares that IQR passes through
4.2	Feature engineering	price, price-per-km, deviation from route median, time-to-departure, hour-of-day, airline one-hot, source
4.3	Self-training pipeline (<code>train_integrity.py</code>)	trained on Kaggle historical + scraped fare corpus, no external API, runs in minutes
4.4	Model artifact + versioning	<code>models/integrity_v1.joblib</code> + metadata (train date, rows, contamination param)
4.5	Inference as pure function	<code>score_integrity(records) → (records + integrity_score + ml_flag, flagged_count)</code>
4.6	IQR + ML dual layer	IQR = transparent hand-checkable baseline, ML = second pass; both logged, nothing silently deleted
4.7	Eval metrics	precision / recall on a labelled synthetic-injection set, printed by the train script

05 AI · Nowcasting / Imputation Model — `pipeline/ml/nowcast.py`

Fills route-days a live scrape missed — imputation, not forecasting. This is how central banks nowcast GDP.

#	FEATURE	DETAIL
5.1	Gradient-boosted regressor	<code>lightgbm</code> / <code>sklearn.GradientBoostingRegressor</code> — fills sparse route-days (a route-day whose scrape failed)
5.2	Features	correlated routes' current prices, route history, day-of-week, days-to-departure, festival flag, fuel-price proxy
5.3	Confidence bands	quantile regression (10th / 90th) or bootstrap → an interval on every imputed value
5.4	NOT forecasting	only imputes missing <i>observed</i> route-days — never predicts future prices. Explicit in console + docs.
5.5	Self-training pipeline (<code>train_nowcast.py</code>)	Kaggle + accumulated scrape data, no API
5.6	Model artifact + versioning	<code>models/nowcast_v1.joblib</code> + metrics (MAE, CI coverage)

#	FEATURE	DETAIL
5.7	Inference as pure function	<code>impute_gaps(route_day_matrix) → (complete_matrix, imputed_mask, confidence_bands)</code>
5.8	Imputation flag on output	run reports <code>imputed_route_days</code> count; index carries a confidence band to the frontend

06 Weighted Laspeyres Engine — `pipeline/laspeyres.py`

#	FEATURE	DETAIL
6.1	Per-route aggregation	pandas <code>groupby("route")["price"].median()</code> (median = outlier-robust) → <code>Pt</code>
6.2	Base-year join	<code>base_year_reference.json</code> → <code>P0</code> , <code>Q0</code> per route, inner join (routes present in both)
6.3	Laspeyres formula	<code>Index = Σ(Pt·Q0) / Σ(P0·Q0) × 100</code>
6.4	Festival decomposition	<code>index_all</code> vs <code>index_ex_festival</code> (re-run on non-flagged fares only) → festival component in index points
6.5	Per-route table returned	<code>Pt, P0, Q0, pt_q0, p0_q0</code> inspectable table → PDF report + readout
6.6	Vectorised, deterministic	no loops, no random seeds — same input = same output
6.7	Golden-number test	<code>test_laspeyres.py</code> — 2-route example = exactly 118.75

07 AI · Explainability Annotator — `pipeline/explain.py`

#	FEATURE	DETAIL
7.1	Rule-based movement annotator	LLM removed — tags each index change with a cause: festival-window proximity, k_factor change, source degradation, imputation share
7.2	Template output	<code>"Index +3.2% vs last run – Diwali window (±7d, 31 fares) + 1 source degraded (Cleartrip→cache)"</code>
7.3	Delta vs previous run	pull last run for same corridor from DB, compute diff, rank drivers
7.4	Pure function	<code>annotate(run, prev_run) → str</code> — no model, no API

08 Persistence — `pipeline/persistence.py`

#	FEATURE	DETAIL
8.1	Postgres primary write	<code>create_engine(PG_DSN, connect_timeout=3)</code> , <code>INSERT</code> into <code>audit_runs</code> , commit
8.2	SQLite WAL fallback	<code>OperationalError</code> / timeout → <code>db/fallback.sqlite</code> , <code>PRAGMA journal_mode=WAL</code> , mirrored schema
8.3	<code>db_target</code> tagging	each row tagged <code>postgres</code> / <code>sqlite_fallback</code> so it can be reconciled later
8.4	<code>audit_runs</code> schema (both DBs)	<code>run_id</code> , <code>corridor</code> , <code>date_range_start/end</code> , <code>index_value</code> , <code>index_ex_festival</code> , <code>k_factor_used</code> , <code>base_period</code> , <code>records_ingested</code> , <code>festival_flagged_count</code> , <code>anomalies_excluded_count</code> , <code>ml_flagged_count</code> , <code>imputed_route_days</code> , <code>scrape_sources (json)</code> , <code>ingestion_source</code> , <code>db_target</code> , <code>created_at</code>
8.5	<code>schema.sql</code>	one DDL file for both databases
8.6	Dual-failure honesty	both fail = disk is gone; an in-memory retry queue is stated future work, not claimed as solved

09 API Contract

#	METHOD · PATH	PURPOSE
9.1	GET <code>/</code>	serve dashboard (<code>static/index.html</code>)
9.2	GET <code>/api/corridors</code>	dropdown options (keys of base ref)
9.3	GET <code>/api/execute-audit</code>	run pipeline, stream logs — <code>text/event-stream</code> (SSE, <code>EventSource</code> -compatible, GET-only)
9.4	GET <code>/api/runs</code>	last N runs for the history chart
9.5	POST <code>/api/import-csv</code>	upload a pre-scraped fare extract → run pipeline on it (badge <code>imported csv</code>)
9.6	GET <code>/api/report/{run_id}</code>	run data for PDF report composition
9.7	SSE line format	<code>data: {"stage","message","run_id",...}\n\n</code> ; final line <code>done:true, index, index_ex_festival, anomalies, sub_index, confidence_band</code>

10 Data Sources & Reference

#	FEATURE	DETAIL
10.1	Kaggle flight-price dataset script	(a) mean 2022 price/route → <code>P0</code> (b) 30–50 rows/corridor → cache seed (c) training corpus for the ML models
10.2	DGCA airport-pair passenger data (data.gov.in)	→ <code>Q0</code> weights — not scraped , official statistic
10.3	<code>base_year_reference.json</code>	<code>{route: {P0, Q0}}</code> , <code>base_period: "2022"</code> , frozen in-repo
10.4	<code>holidays</code> package	correct lunar dates, offline
10.5	Travelpayouts affiliate token	free signup, <code>.env</code> , grants fallback API access
10.6	Scrape-target recon doc	per source: search-URL pattern + fare XHR endpoint (DevTools → Network)
10.7	CPI reference series	real MoSPI CPI "Transport" monthly 2022→2026 as a static array (chart overlay) — never a synthetic ramp

11 Tests & Ship

#	FEATURE	DETAIL
11.1	<code>test_iqr.py</code>	golden numbers, hand-calculated
11.2	<code>test_laspeyres.py</code>	golden numbers = 118.75
11.3	<code>test_festival.py</code>	2026 dates, ±7d boundary
11.4	<code>test_e2e.py</code>	full pipeline on cache / backup data
11.5	<code>test_scrape_fallback.py</code>	network kill → Travelpayouts → cache chain, each tier logged
11.6	ML model eval scripts	integrity precision / recall, nowcast MAE + CI coverage
11.7	<code>run.py</code>	starts uvicorn (pywebview optional / skipped — browser console is the shell)
11.8	<code>requirements.txt</code> + <code>.env.example</code>	fastapi, uvicorn, httpx, playwright, pandas, numpy, scikit-learn, lightgbm, holidays, sqlalchemy, pydantic-settings, apscheduler

★ Fallback story — for the judges

SAY THIS

Primary: live scraping of 5 airline / OTA portals — real headless browsers that intercept each site's own fare XHR.

Everything is live — that is what makes us different.

Tier 1 fallback: a source is blocked → the **Travelpayouts (Aviasales) data API** supplies that route's fares.

Tier 2 fallback: fully offline → the last cached scrape snapshot.

Every run reports which tier was used (`ingestion_source`). DGCA volume weights are never scraped — that is an official statistic.

WHAT CHANGED FROM EARLIER DRAFTS

Self-healing vision-LLM extractor (1.5c) — ~~removed~~, replaced by a CSS-selector fallback.

Free LLM API key — ~~removed~~. No external AI APIs anywhere.

Explainability LLM — ~~removed~~, replaced by a rule-based template annotator.

`pywebview` desktop shell — skipped; the browser console is the shell (note: PS says "desktop app", so keep `pywebview` as a ~15-line stretch goal if time allows).

> Build order (do not skip ahead)

Engineering deep-dive §10. Tell the tool: *"only build stage N, don't implement later stages."*

- **1 Scaffold** — folders, stub functions with correct signatures, `requirements.txt`, `.env.example`, `config.py`
- **2 Cache-seeded fallback** — seed `data/cache/` from Kaggle, `fetch_fares` returns it
- **3 SSE bridge** — real `run_audit` generator, hardcoded lines, wired to `EventSource`
- **4 Festival engine** → **5 IQR filter** → **6 Laspeyres** (each with its golden-number test)
- **7 Live scraper pool** — one adapter, then cache layer, then Travelpayouts fallback, then fan-out to all 5
- **8 Persistence** — Postgres (Docker) + SQLite fallback + `schema.sql`
- **9 AI models** — train integrity + nowcast offline, wire inference as pure functions
- **10 Explainability annotator + polish** — `/api/runs`, CSV import, PDF report data